

API Detección de Construcciones

Documentación técnica para desarrolladores

Motor GPU · 10.0.0.30:8100 · 2026-09-11

GAMC · Módulo IDEC ERP · Pipeline ECC + BTC/SAM2 + SISCAT

Audiencia: desarrolladores que integran el módulo (ERP back/front, clientes HTTP, QA).

Alcance: contratos HTTP del motor en 10.0.0.30:8100 y el espejo autenticado del ERP en /api/deteccion/*.

1. Arquitectura y despliegue

El motor de detección corre en el servidor GPU **10.0.0.30** (servicio systemd catastro-deteccion-api, Uvicorn FastAPI puerto **8100**). El ERP IDEC no ejecuta BTC/SAM2 localmente: el backend idec-erp-back actúa como **proxy autenticado** hacia el motor.

Browser → ERP Back /api/deteccion/* → http://10.0.0.30:8100/api/v1/*
 (Bearer JWT + permisos) (X-API-Key opcional)
 Imágenes: GET /api/deteccion/motor/{asset_path} → GET /outputs/... en GPU

Componente	Ubicación / URL
Motor API	http://10.0.0.30:8100
Swagger / ReDoc	http://10.0.0.30:8100/docs
OpenAPI JSON	http://10.0.0.30:8100/openapi.json
Código servidor	/usr/src/catastro_deteccion_construcciones
Unit systemd	catastro-deteccion-api.service
WMS ortofotos	https://gs.catastrocbba.com
ERP proxy prefix	/api/deteccion

Importante: en producción ERP el front solo habla con el back. No exponer :8100 al navegador. Las URLs de imágenes del resultado (/outputs/...) se reescriben al proxy /api/deteccion/motor/....

2. Autenticación y permisos

Motor GPU (10.0.0.30)

Si la variable de entorno CATASTRO_API_KEY está definida en el unit systemd, todas las rutas (salvo estáticos según configuración) exigen:

X-API-Key: <secreto>

Consultar si está activa: GET /health → api_key_required.

ERP Back (proxy)

Permiso	Operaciones
deteccion.ver	salud, capas WMS, progreso, resultado, alineación GET, registro SISCAT
deteccion.editar	iniciar detección, cancelar, preview/apply alineación manual
usuario autenticado	proxy de assets /motor/{path}

3. Mapeo ERP ↔ Motor GPU

El front consume solo rutas ERP. El back reenvía al motor. Tabla de equivalencia:

ERP (idec-erp-back)	Motor GPU :8100	Método
/api/deteccion/salud	/health	GET
/api/deteccion/wms/capas	/api/v1/wms/layers	GET
/api/deteccion/trabajos/detectar-wms	/api/v1/detect-changes-wms-async	POST
/api/deteccion/trabajos/{id}/progreso	/api/v1/jobs/{id}/progress	GET
/api/deteccion/trabajos/{id}/resultado	/api/v1/jobs/{id}/result	GET
/api/deteccion/trabajos/{id}/cancelar	/api/v1/jobs/{id}/cancel	POST
/api/deteccion/trabajos/{id}/alineacion-manual	/api/v1/jobs/{id}/align-manual	GET
.../alineacion-manual/vista-previa	.../align-manual/preview	POST
.../alineacion-manual/aplicar	.../align-manual/apply	POST

ERP (idec-erp-back)	Motor GPU :8100	Método
/api/deteccion/catastro/registro-catastral/{id}	/api/v1/catastro/registro-catastral/{id}	GET
/api/deteccion/motor/{path}	{path} (p.ej. /outputs/...)	GET

4. Flujo operativo recomendado

- 1) GET /salud → motor reachable + GPUs
- 2) GET /wms/capas → años / hosts WMS
- 3) POST /trabajos/detectar-wms → { job_id }
- 4) GET /trabajos/{job_id}/progreso → poll cada 1-2 s hasta status=done|error|cancelled
- 5) GET /trabajos/{job_id}/resultado → hallazgos + align_quality + view_geometry + urls
- 6) Si align poor/bad → puntos GCP → preview → apply (re-detecta)
- 7) GET /catastro/registro-catastral/{idRegistro} → ficha SISCAT

Timeouts sugeridos en cliente: progreso 15–30 s; resultado 60–180 s; inicio de job 30–60 s. El pipeline completo (WMS+align+BTC) puede tardar varios minutos en áreas grandes (tiles).

5. Catálogo de endpoints del motor (10.0.0.30:8100)

Inventario generado desde OpenAPI en vivo. Detalle de los endpoints críticos para el módulo de detección WMS.

5.1 Salud e infraestructura

GET /health

Estado del servicio, CUDA y GPUs. Sin autenticación típica.

```
{
  "status": "ok",
  "cuda": true,
  "gpus": [{"index": 0, "name": "NVIDIA GeForce RTX 3060"}, ...],
  "api_key_required": false
}
```

GET / · GET /ui

UI HTML de prueba del motor (no usar en ERP producción).

5.2 Catálogo WMS y planificación

GET /api/v1/wms/layers

Descubre capas anuales en ArcGIS (<https://gs.catastrocbba.com>). Devuelve hosts, years, layers (mapa año→servicio).

POST /api/v1/wms/resolution-plan

Calcula ancho/alto/GSD/tiles para un bbox sin ejecutar detección. Body: bbox, target_gsd_m (def. 0.30), max_px, topes de tile (~260 m / ~6.71 ha).

5.3 Detección de cambios (núcleo del producto)

POST /api/v1/detect-changes-wms-async — endpoint principal para ERP

Encola el pipeline: descarga WMS A/B → alineación → sombras (opcional) → BTC/SAM2 → cruce predios/asientos → reporte. Responde de inmediato con job_id. El cliente hace polling.

Body JSON (campos principales):

Campo	Tipo	Default / notas
year_ref	int	Año A (referencia) — requerido
year_mov	int	Año B (comparación) — requerido
bbox	float[4]	CRS:84 [minLon,minLat,maxLon,maxLat]
polygon	float[][2]	Anillo [[lon,lat],...] ≥3; filtra fuera del polígono
target_gsd_m	float	0.30 m/px (nitidez techos)
max_px	int	4096 tope por tile
min_area_m2	float	área mínima construcción (m²)
min_prob_pct	float	umbral score 0–100 para alertas
predios_buffer_m	float	buffer cruce predio (m)
gpu	int	0 o 1
min_diff	float	umbral diferencia radiométrica

```
curl -sS -X POST http://10.0.0.30:8100/api/v1/detect-changes-wms-async \
-H 'Content-Type: application/json' \
-d '{
  "year_ref": 2018,
  "year_mov": 2024,
  "bbox": [-66.1575, -17.3945, -66.1550, -17.3920],
  "target_gsd_m": 0.30,
  "min_prob_pct": 40,
  "predios_buffer_m": 10,
  "gpu": 0
}'
```

Respuesta inmediata típica:

```
{ "job_id": "wms_chg_YYYYMMDD_..._abcd", "status": "running" }
```

POST /api/v1/detect-changes-wms

Misma lógica que async pero **síncrona** (bloquea hasta terminar). Útil para scripts; no recomendado para UI (timeouts).

POST /api/v1/detect-changes

Pipeline con archivos locales (multipart: ref, mov). No descarga WMS.

5.4 Jobs: progreso, resultado, cancelación**GET /api/v1/jobs/{job_id}/progress**

Campos típicos: status (running|done|error|cancelled), pct, step, step_label, detail, steps[], n_tiles, phase_info (p.ej. sombras).

GET /api/v1/jobs/{job_id}/result

Disponibile cuando status=done. Si aún corre → HTTP 409. Si falló → 500 con detalle.

Campos clave del resultado:

Campo	Significado
job_id	Identificador del trabajo
n_nueva / n_eliminada / n_cambio	Conteos de alertas $\geq$ umbral
cambios / reporte_arquitecto	Lista de hallazgos (+ código catastral / asientos)
align_quality	level good warn poor bad, residual_m, message
view_geometry	Heurística oblicuidad: level, warned, score_a/b, message
needs_manual_align	true si align poor/bad
urls / vista	Rutas relativas a overlays e imágenes alineadas
img_bbox, img_width, img_height	Georreferencia de la imagen resultado
resumen_confiabilidad	Mensaje usuario + conteos confirmadas/revisar

URLs de imagen: son relativas al motor (/outputs/{job_id}/result/...). En ERP se reescriben a /api/deteccion/motor/outputs/....

POST /api/v1/jobs/{job_id}/cancel

Solicita cancelación cooperativa del job en curso.

GET /api/v1/jobs/{job_id}/files/{filename}

Descarga un archivo concreto del outdir del job.

DELETE /api/v1/jobs/{job_id}

Elimina artefactos del job en disco.

5.5 Alineación manual (GCP)**GET /api/v1/jobs/{job_id}/align-manual**

Metadatos / estado para UI de puntos de control.

POST .../align-manual/preview

Estima warp sin re-detectar. Body: points: [{ax,ay,bx,by}, ...] (≥ 3), method: affine|homography. Respuesta incluye residual_m e imagen de vista previa.

POST .../align-manual/apply

Aplica warp y **re-ejecuta detección**. Mismo body que preview. Si residual es excesivo → HTTP 400.

5.6 SISCAT / registro catastral**GET /api/v1/catastro/registro-catastral/{id_registro_catastral}**

Expediente de un asiento (solo lectura): registro catastral, informes legal/técnico, propietarios, construcciones. El `id` proviene de `idRegistroCatastral` en el cruce de asientos del resultado. **No exponer IDs internos en UI de negocio** si la política de producto lo prohíbe; sí son necesarios para esta llamada API.

5.7 Endpoints auxiliares (laboratorio)

Endpoint	Uso
POST /api/v1/align	Solo ECC sobre archivos ref/mov
POST /api/v1/align-wms	ECC descargando WMS por años
POST /api/v1/segment	SAM2 puntual/caja
POST /api/v1/segformer	Landcover SegFormer archivo
POST /api/v1/segformer-wms	Landcover SegFormer WMS

Estos endpoints son útiles para depuración; el flujo de producto ERP usa **detect-changes-wms-async** + jobs + alineación manual.

6. Semántica de negocio en el resultado

6.1 Tipos de cambio

tipo	Significado operativo
nueva	Aparece en B y no en A (posible obra nueva)
eliminada	Estaba en A y no en B
cambio	Existe en ambos años con huella/área distinta
sin_cambio	Emparejada sin cambio relevante (suele filtrarse del overlay)

6.2 Confiabilidad de alineación (align_quality.level)

level	Acción recomendada en UI
good / ok	Continuar validación fiscal habitual
warn	Revisar tablero A/B; ofrecer alineación manual
poor / bad	Bloquear confianza: exigir GCP + apply

6.3 Oblicuidad (view_geometry)

Heurística (no ortorrectifica). Si warned=true, la UI debe advertir que A/B pueden tener geometría de captura distinta (fachadas vs techos). Niveles: ok | warn | high | unknown.

7. Códigos HTTP y errores

Código	Cuándo
200	OK
400	Payload inválido, residual GCP excesivo, bbox inválido
401	API key inválida (motor) / JWT inválido (ERP)
403	Sin permiso deteccion.ver / deteccion.editar (ERP)
404	Job o recurso inexistente
409	Resultado pedido mientras el job aún corre
500	Error interno / job en error
503	Motor GPU no reachable (ERP proxy)

8. Ejemplos de uso

Ejemplos listos para copiar. Sustituya años, bbox y TOKEN según su entorno. Base motor: `http://10.0.0.30:8100`. Base ERP: la URL pública de `idec-erp-back` (prefijo `/api/deteccion`).

8.1 Salud del motor y capas WMS (curl)

```
# Salud + GPUs
curl -sS http://10.0.0.30:8100/health | jq

# Si CATASTRO_API_KEY está activa:
# curl -sS -H 'X-API-Key: SECRETO' http://10.0.0.30:8100/health | jq

# Años / hosts WMS (gs.catastrocbba.com)
curl -sS http://10.0.0.30:8100/api/v1/wms/layers | jq '{hosts, count, years: .years[:8]}'
```

8.2 Plan de resolución (sin detectar)

```
curl -sS -X POST http://10.0.0.30:8100/api/v1/wms/resolution-plan \
-H 'Content-Type: application/json' \
-d '{
  "bbox": [-66.1575, -17.3945, -66.1550, -17.3920],
  "target_gsd_m": 0.30,
  "max_px": 4096,
  "min_area_m2": 80
}' | jq '{mode, width, height, n_tiles, effective_gsd_m, note}'
```

8.3 Detección async completa (curl + polling)

Flujo productivo: iniciar job → consultar progreso → leer resultado. Áreas grandes pueden tardar varios minutos (tiles).

```
BASE=http://10.0.0.30:8100

JOB=$(curl -sS -X POST "$BASE/api/v1/detect-changes-wms-async" \
-H 'Content-Type: application/json' \
-d '{
  "year_ref": 2018,
  "year_mov": 2024,
  "bbox": [-66.1575, -17.3945, -66.1550, -17.3920],
  "target_gsd_m": 0.30,
  "min_prob_pct": 40,
  "predios_buffer_m": 10,
  "gpu": 0
}' | jq -r .job_id)

echo "job_id=$JOB"

while true; do
  PROG=$(curl -sS "$BASE/api/v1/jobs/$JOB/progress")
  echo "$PROG" | jq '{status, pct, step_label, detail}'
  ST=$(echo "$PROG" | jq -r .status)
  case "$ST" in done|error|cancelled) break ;; esac
  sleep 2
done

curl -sS "$BASE/api/v1/jobs/$JOB/result" | jq '{
  n_nueva, n_eliminada, n_cambio,
  align: .align_quality.level,
  residual_m: .align_residual_m,
  oblicuidad: .view_geometry,
  urls: .urls
}'
```

8.4 Detección con polígono de selección

Si el usuario dibuja un polígono en el mapa, envíe `polygon` (anillo lon/lat). El motor descarga el bbox envolvente y filtra hallazgos fuera del polígono.

```
curl -sS -X POST http://10.0.0.30:8100/api/v1/detect-changes-wms-async \
-H 'Content-Type: application/json' \
-d '{
  "year_ref": 2010,
```

```

    "year_mov": 2023,
    "polygon": [
      [-66.1570, -17.3940],
      [-66.1555, -17.3940],
      [-66.1555, -17.3928],
      [-66.1570, -17.3928],
      [-66.1570, -17.3940]
    ],
    "min_prob_pct": 50,
    "target_gsd_m": 0.30
  }'

```

8.5 Cancelar un job en curso

```

curl -sS -X POST http://10.0.0.30:8100/api/v1/jobs/$JOB/cancel \
-H 'Content-Type: application/json' -d '{}'
curl -sS http://10.0.0.30:8100/api/v1/jobs/$JOB/progress | jq '{status, step_label}'

```

8.6 Alineación manual (preview y apply)

Coordenadas en píxeles de las ortofotos A (referencia) y B (móvil). Mínimo 3 pares para affine; 4+ para homography.
apply re-ejecuta la detección.

```

# Vista previa (no re-detecta)
curl -sS -X POST "http://10.0.0.30:8100/api/v1/jobs/$JOB/align-manual/preview" \
-H 'Content-Type: application/json' \
-d '{
  "method": "affine",
  "points": [
    {"ax": 120, "ay": 80, "bx": 125, "by": 82},
    {"ax": 400, "ay": 90, "bx": 405, "by": 93},
    {"ax": 200, "ay": 300, "bx": 204, "by": 305},
    {"ax": 520, "ay": 410, "bx": 528, "by": 418}
  ]
}' | jq '{residual_m, residual_px, method}'

# Aplicar y re-detectar
curl -sS -X POST "http://10.0.0.30:8100/api/v1/jobs/$JOB/align-manual/apply" \
-H 'Content-Type: application/json' \
-d '{
  "method": "affine",
  "points": [
    {"ax": 120, "ay": 80, "bx": 125, "by": 82},
    {"ax": 400, "ay": 90, "bx": 405, "by": 93},
    {"ax": 200, "ay": 300, "bx": 204, "by": 305},
    {"ax": 520, "ay": 410, "bx": 528, "by": 418}
  ]
}' | jq '{job_id, residual_m, status}'

```

8.7 Expediente SISCAT (registro catastral)

El idRegistroCatastral sale del cruce de asientos en el resultado (no confundir con código catastral visible en UI).

```

ID_REG=123456
curl -sS "http://10.0.0.30:8100/api/v1/catastro/registro-catastral/$ID_REG" \
| jq '{asiento: .registro_catastral.asiento,
      nroInscripcion: .registro_catastral.nroInscripcion,
      legales: (.informes_legales|length),
      tecnicos: (.informes_tecnicos|length)}'

```

8.8 Descargar / mostrar imágenes del resultado

```

# Motor directo (laboratorio)
curl -sS -o overlay.png \
  "http://10.0.0.30:8100/outputs/$JOB/result/cambios_overlay.png"

# 0 vía endpoint de archivos
curl -sS -o aligned_b.png \
  "http://10.0.0.30:8100/api/v1/jobs/$JOB/files/mov_aligned.png"

# ERP (producción): el front usa el proxy autenticado
# GET /api/deteccion/motor/outputs/<job>/result/cambios_overlay.png
curl -sS -H "Authorization: Bearer $TOKEN" -o overlay.png \
  "https://<erp-host>/api/deteccion/motor/outputs/$JOB/result/cambios_overlay.png"

```

8.9 Cliente JavaScript / TypeScript (ERP)

Patrón usado por el front IDEC: Bearer JWT + rutas /api/deteccion/*.

```
const ERP = 'https://<erp-host>'; // idec-erp-back
const headers = {
  Authorization: `Bearer ${token}`,
  'Content-Type': 'application/json',
};

// 1) Salud
const salud = await fetch(`${ERP}/api/deteccion/salud`, { headers })
  .then(r => r.json());
if (!salud.reachable) throw new Error('Motor GPU no disponible');

// 2) Iniciar detección
const started = await fetch(`${ERP}/api/deteccion/trabajos/detectar-wms`, {
  method: 'POST', headers,
  body: JSON.stringify({
    year_ref: 2018,
    year_mov: 2024,
    bbox: [-66.1575, -17.3945, -66.1550, -17.3920],
    target_gsd_m: 0.30,
    min_prob_pct: 40,
    predios_buffer_m: 10,
    gpu: 0,
  }),
}).then(r => r.json());

const jobId = started.job_id;

// 3) Poll progreso
async function waitDone(id) {
  for (;;) {
    const p = await fetch(`${ERP}/api/deteccion/trabajos/${id}/progreso`, { headers })
      .then(r => r.json());
    if (['done', 'error', 'cancelled'].includes(p.status)) return p;
    await new Promise(r => setTimeout(r, 1500));
  }
}
const prog = await waitDone(jobId);
if (prog.status !== 'done') throw new Error(prog.error || prog.detail || prog.status);

// 4) Resultado
const result = await fetch(`${ERP}/api/deteccion/trabajos/${jobId}/resultado`, { headers })
  .then(r => r.json());

console.log(result.n_nueva, result.align_quality?.level, result.view_geometry?.warned);

// 5) URL de imagen vía proxy (el motor no se expone al browser)
const overlayPath = (result.urls?.overlay || '').replace(/\/\+/g, '');
const overlayUrl = `${ERP}/api/deteccion/motor/${overlayPath}`;
// <img src={overlayUrl} /> con el mismo Bearer (o blob vía fetch)
```

8.10 Cliente Python (requests)

```
import time, requests

BASE = 'http://10.0.0.30:8100'
s = requests.Session()
# s.headers['X-API-Key'] = 'SECRETO' # si aplica

r = s.post(f'{BASE}/api/v1/detect-changes-wms-async', json={
  'year_ref': 2018,
  'year_mov': 2024,
  'bbox': [-66.1575, -17.3945, -66.1550, -17.3920],
  'target_gsd_m': 0.30,
  'min_prob_pct': 40,
  'gpu': 0,
}, timeout=60)
r.raise_for_status()
job_id = r.json()['job_id']

while True:
  p = s.get(f'{BASE}/api/v1/jobs/{job_id}/progress', timeout=30).json()
  print(p.get('pct'), p.get('step_label'), p.get('status'))
```

```

        if p.get('status') in ('done', 'error', 'cancelled'):
            break
        time.sleep(2)

    if p.get('status') != 'done':
        raise RuntimeError(p)

    result = s.get(f'{BASE}/api/v1/jobs/{job_id}/result', timeout=180).json()
    print('nuevas', result.get('n_nueva'), 'align', (result.get('align_quality') or {}).get('level'))
    for row in (result.get('reporte_arquitecto') or [])[5:]:
        print(row.get('tipo'), row.get('prob_pct'), row.get('codigo_catastral'))

```

8.11 Detección síncrona con archivos locales (laboratorio)

No usa WMS: sube dos PNG/JPG. Útil para pruebas offline. Timeout largo recomendado (3–5 min).

```

curl -sS -m 300 -X POST http://10.0.0.30:8100/api/v1/detect-changes \
-F 'ref=@./orto_2018.png' \
-F 'mov=@./orto_2024.png' \
-F 'gpu=0' \
-F 'min_area=1500' \
-F 'min_diff=30' | jq '{job_id, n_nueva, n_eliminada, n_cambio, ecc_cc}'

```

Nota ERP: en producción prefiera siempre el flujo async vía `/api/deteccion/trabajos/detectar-wms`. El endpoint síncrono y el acceso directo a `:8100` son para laboratorio/ops.

9. Operación en el servidor GPU

```

systemctl status catastro-deteccion-api
systemctl restart catastro-deteccion-api
journalctl -u catastro-deteccion-api -f
curl -s http://127.0.0.1:8100/health | jq
curl -s http://127.0.0.1:8100/api/v1/wms/layers | jq '.hosts, .years[:5], .count'

```

10. Inventario OpenAPI (snapshot)

Título: **Catastro Detección Construcciones** · Versión API: **0.6.0** · Rutas: **22**

Método	Ruta
GET	/
POST	/api/v1/align
POST	/api/v1/align-wms
GET	/api/v1/catastro/registro-catastral/{id_registro_catastral}
POST	/api/v1/detect-changes
POST	/api/v1/detect-changes-wms
POST	/api/v1/detect-changes-wms-async
DELETE	/api/v1/jobs/{job_id}
GET	/api/v1/jobs/{job_id}/align-manual
POST	/api/v1/jobs/{job_id}/align-manual/apply
POST	/api/v1/jobs/{job_id}/align-manual/preview
POST	/api/v1/jobs/{job_id}/cancel
GET	/api/v1/jobs/{job_id}/files/{filename}
GET	/api/v1/jobs/{job_id}/progress
GET	/api/v1/jobs/{job_id}/result
POST	/api/v1/segformer
POST	/api/v1/segformer-wms
POST	/api/v1/segment
GET	/api/v1/wms/layers
POST	/api/v1/wms/resolution-plan

Método	Ruta
GET	/health
GET	/ui

Documento generado automáticamente para el equipo de desarrollo IDEC/GAMC. Fuente de verdad adicional: Swagger en <http://10.0.0.30:8100/docs>